Plano de Estudo: Estruturas de Dados e Algoritmos

Disciplina: Algoritmos e Estruturas de Dados

Bibliografia Base: Cormen, T.H. et al. *Algoritmos: Teoria e Prática*.

---

1. O Pilar da Organização: Listas e Filas

Conceito: Como armazenar dados linearmente quando não sabemos o tamanho final do conjunto.

Estrutura: Lista Ligada (Linked List)

- **O que é:** Uma sequência de nós onde cada um aponta para o próximo.
- **Por que usar:** Inserção e remoção rápidas sem precisar deslocar todos os outros elementos (como ocorre no Array).

Código (Baseado no Cormen - LIST-INSERT):

Plaintext

```
LIST-INSERT(L, x)
1  x.next = L.head
2  se L.head != NIL
3      então L.head.prev = x
4  L.head = x
5  x.prev = NIL
```

---

2. O Pilar da Eficiência: Árvores Binárias de Busca (BST)

Conceito: Organizar dados de forma hierárquica para que a busca seja extremamente veloz.

Estrutura: Árvore Binária

- **O que é:** Cada "pai" tem dois "filhos". À esquerda ficam os menores, à direita os maiores.

- **Por que usar:** Reduz o tempo de busca drasticamente. Em vez de olhar item por item, você descarta metade da árvore a cada passo.

Código (Baseado no Cormen - TREE-INSERT):

Plaintext

```
TREE-INSERT(T, z)
1  y = NIL
2  x = T.root
3  enquanto x != NIL
4      y = x
5      se z.key < x.key
6          então x = x.left
7          senão x = x.right
8  z.p = y
9  se y == NIL
10     então T.root = z
11     senão se z.key < y.key
12         então y.left = z
13         senão y.right = z
```

---

3. O Pilar da Prioridade: Heaps

Conceito: Gerenciar um conjunto de dados onde o elemento mais importante deve estar sempre acessível.

Estrutura: Max-Heap (implementado em Array)

- **O que é:** Uma árvore completa mapeada dentro de um vetor comum.
- **Por que usar:** É a base para filas de prioridade (ex: triagem de hospital ou processos de CPU).

Código (Baseado no Cormen - MAX-HEAPIFY):

Plaintext

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  se l <= A.heap-size e A[l] > A[i]
4      então maior = l
5      senão maior = i
6  se r <= A.heap-size e A[r] > A[maior]
7      então maior = r
8  se maior != i
9      então trocar A[i] com A[maior]
10           MAX-HEAPIFY(A, maior)
```

4. O Pilar da Conexão: Grafos

Conceito: Representar relacionamentos complexos (redes sociais, rotas de entrega).

Estrutura: Lista de Adjacência

- **O que é:** Um array onde cada índice representa um ponto (vértice) e contém uma lista de seus vizinhos.
- **Algoritmo de Estudo:** BFS (Busca em Largura) para achar a menor distância.

Código (Baseado no Cormen - BFS):

Plaintext

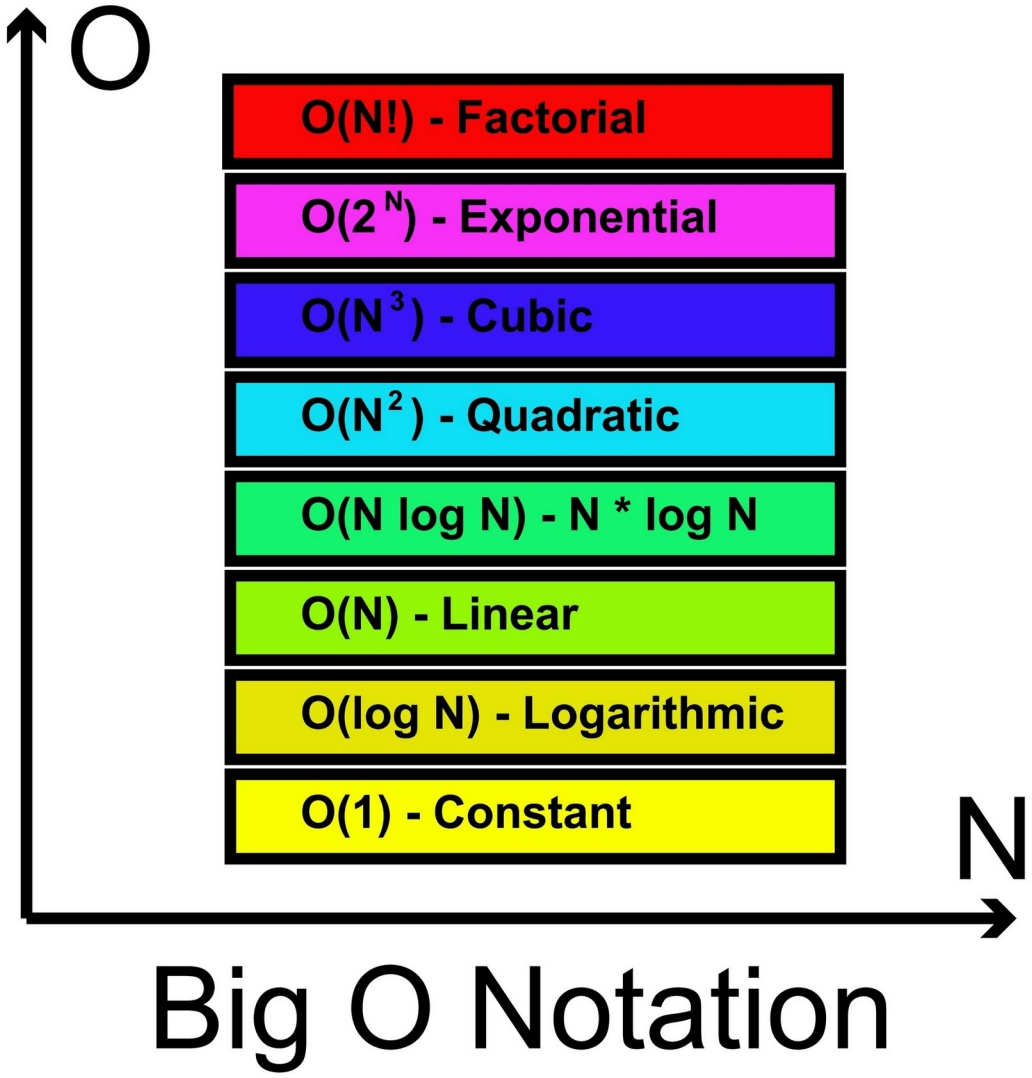
```
BFS(G, s)
para cada vértice u em G.V - {s}
    u.cor = BRANCO
s.cor = CINZA
Q = FILA-VAZIA()
ENQUEUE(Q, s)
enquanto Q não estiver vazia
    u = DEQUEUE(Q)
    para cada v em G.Adj[u]
        se v.cor == BRANCO
            v.cor = CINZA
            ENQUEUE(Q, v)
    u.cor = PRETO
```

Aqui está o seu **Compêndio de Estudo de Algoritmos (Cormen)**:

---

1. Análise e Fundamentos (O Alicerce)

Antes dos códigos, entenda as curvas de crescimento. Isso é o que permite você saber se seu código é "bom" ou "ruim".



Shutterstock
Explorar

---

2. Ordenação (O Treino da Lógica)

Aqui você aprende a manipular arrays e a estratégia de **Divisão e Conquista**.

Merge Sort (Divisão e Conquista)

Plaintext

```
MERGE-SORT(A, p, r)
1  se p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

Quick Sort (O mais usado na prática)

Plaintext

```
QUICKSORT(A, p, r)
1  se p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q - 1)
4      QUICKSORT(A, q + 1, r)
```

---

3. Estruturas Elementares (O Coração)

Como organizar dados de forma linear ou por mapeamento.

Pilhas (Stack - LIFO) e Filas (Queue - FIFO) O livro foca nas operações de PUSH, POP (Pilha) e ENQUEUE, DEQUEUE (Fila).

Tabela Hash (A estrutura mais importante)

Plaintext

```
CHAINED-HASH-SEARCH(T, k)
1  procurar k na lista T[h(k)]
```

---

4. Árvores (Estrutura de Decisão)

Como organizar dados para busca rápida não-linear.

Busca em Árvore Binária (BST)

Plaintext

```
TREE-SEARCH(x, k)
1  se x == NIL ou k == x.key
2      então retorne x
3  se k < x.key
```

```
4      então retorne TREE-SEARCH(x.left, k)
5      senão retorne TREE-SEARCH(x.right, k)
```

---

5. Grafos (O Mapa do Mundo)

Como conectar pontos e achar o melhor caminho.

Busca em Largura (BFS)

Plaintext

```
BFS(G, s)
1  para cada vértice u ∈ G.V - {s}
2      u.cor = BRANCO; u.d = ∞; u.p = NIL
3  s.cor = CINZA; s.d = 0; s.p = NIL
4  Q = Ø
5  ENQUEUE(Q, s)
6  enquanto Q ≠ Ø
7      u = DEQUEUE(Q)
8      para cada v ∈ G.Adj[u]
9          se v.cor == BRANCO
10             v.cor = CINZA; v.d = u.d + 1; v.p = u
11             ENQUEUE(Q, v)
12     u.cor = PRETO
```

Algoritmo de Dijkstra (Caminho mais curto)

Plaintext

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = Ø
3  Q = G.V
4  enquanto Q ≠ Ø
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      para cada vértice v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

---

6. Programação Dinâmica (Nível Mestre)

Como não refazer trabalho que já foi feito. O exemplo clássico é o corte de hastes ou a sequência de Fibonacci otimizada.

Corte de Haste (Bottom-up)

Plaintext

```
BOTTOM-UP-CUT-ROD(p, n)
```

```
seja r[0..n] um novo arranjo
r[0] = 0
para j = 1 até n
    q = -∞
    para i = 1 até j
        q = max(q, p[i] + r[j - i])
    r[j] = q
retorne r[n]
```

Dicas Finais para se tornar um Programador Melhor:

1. **Não apenas copie:** Tente explicar para si mesmo o que cada linha faz.
2. **Implemente:** Escolha uma linguagem (como Python ou C) e tente fazer o **Merge Sort** funcionar nela.
3. **Use o PDF:** Sempre que tiver dúvida em um passo do pseudocódigo, vá ao capítulo indicado no livro. O Cormen explica a "prova de correção" de cada um deles.

Este conjunto de algoritmos é o que diferencia um "codificador" de um **Engenheiro de Software**. Bom estudo para o seu Projeto Integrador!